Reproducible and Portable Scientific Computing on HPC with Containers

Author: Yucheng Zhang, Senior Bioinformatic Engineer, TTS Research Technology, yucheng.zhang@tufts.edu

Date: 2026-02-20

Introduction

Containers

- **Container**: A container is an abstraction for a set of technologies that aim to solve the problem of how to get software to run reliably when moved from one computing environment to another.
- **Image**: A container image is simply a file (or collection of files) saved on disk that stores everything you need to run a target application or applications.
- **Registry**: a place to store (and share) container images.

Why use containers?

- **Getting organized**: containers keep things organized by isolating programs and their dependencies inside containers.
- **Build once, run almost anywhere**: containers allow us to package up our complete software environment and ship it to other computing environments.
- **Reproducibility**: containers can ensure identical versions of apps, libraries, compilers, etc.

Docker

The concept of containers emerged in the 1970s, but it did not gain widespread attention until the introduction of Docker in 2013. Docker is an open-source platform for building, deploying, and managing containerized applications.

HPC systems generally avoid Docker because it requires root privileges, which poses security risks on shared clusters. In addition, Docker’s daemon-based model does not integrate well with HPC schedulers and multi-user environments, making alternatives like Singularity/Apptainer more suitable.

Singularity

Singularity was developed in 2015 as an open-source project by researchers at Lawrence Berkeley National Laboratory (LBNL) led by Gregory Kurtzer.

Singularity has become the containerization framework of choice in HPC environments.

1. Enable researchers to package entire scientific workflows, libraries, and even data.
2. Can use docker images.
3. Does not require root privileges to run.

Apptainer

In 2021, the Singularity project split. The open-source version was moved to the **Linux Foundation** and renamed **Apptainer**. Meanwhile, a commercial entity (Sylabs) continued to develop a version called **SingularityCE** (Community Edition).

Because Apptainer is the successor to the original Singularity project, they are highly compatible. You can typically use `apptainer` commands to run `.sif` files created with Singularity.

Which one to use?

Our cluster supports both Singularity and Apptainer. Please note the different prefixes required to pass environment variables into your containers:

- Singularity: Use **SINGULARITYENV_**
- Apptainer: Use **APPTAINERENV_**

Although Apptainer can still process the **SINGULARITYENV_** prefix as a fallback,

users are encouraged to use the **APPTAINERENV_** prefix to ensure their scripts remain compatible with future updates.

Comparison: Singularity vs. Apptainer

Feature	Singularity (CE/Sylabs)	Apptainer (Linux Foundation)
Project Origin	Continued by Sylabs after the 2021 split.	Open-source branch moved to the Linux Foundation.
Command	`singularity`	`apptainer` (though `singularity` often works as an alias).
Env Prefix	`SINGULARITYENV_<VAR>`	`APPTAINERENV_<VAR>`.
Compatibility	Native SIF support.	Supports SIF and legacy `SINGULARITYENV_` prefixes.
Registry	Native support for Sylabs Cloud (`library://`).	Highly compatible with Docker Hub and GitHub Container Registry.

Registries

A container registry is a centralized platform where container images are stored, managed, and distributed. It acts like a repository for software containers, similar to how GitHub is used for managing source code. Container registries allow users to pull (download) pre-built container images or push (upload) their own images for sharing or deployment across various environments, including HPC clusters, cloud platforms, and local machines.

- Docker Hub: The largest and most common registry. Most Apptainer/Singularity users pull from here using `docker://`.
- GitHub Container Registry (ghcr.io): Increasingly popular for open-source projects because it integrates directly with GitHub Actions and repositories.

- Quay.io: Owned by Red Hat; known for its robust security scanning and history of being a stable alternative to Docker Hub.
- Sylabs Cloud (Library): The native registry for Singularity images. It hosts .sif files directly. Accessed via `library://`.
- BioContainers: A massive community project that provides containers specifically for bioinformatics tools. Most are hosted on Quay.io or Docker Hub.
- NVIDIA GPU Cloud (NGC): Essential if your users are doing Machine Learning or GPU-accelerated work (PyTorch, TensorFlow, etc.). It contains highly optimized containers for NVIDIA hardware.

Sub-commands

- pull: download a container from a given URI.
- run: executes the default `runscript` defined within the container image.
- exec: run a specific command or script inside the container, bypassing the default `runscript`.
- shell: opens an interactive terminal (shell) inside the container. This is excellent for debugging or exploring the container environment.
- build: create a .sif image from a definition file (.def), a Docker URL, or another Singularity image.
- inspect: displays metadata about the image, such as its definition file, environment variables, and labels.

> 💡 **Concept:** Use `run` for the container's default intended purpose. Use `exec` when you want to run a specific command (like `python` or `ls`) inside the container.

Hands-on

Load modules

It is highly recommended to run `module purge` before loading singularity.

In [ ]:
```
module purge
module avail apptainer
module avail singularity
```

> **ℹ Note on Software Choice: Apptainer vs. Singularity**
> While the Tufts HPC supports both `singularity` and `apptainer`, and Apptainer is the generally recommended open-source path forward, we will use the `singularity` command for this workshop. This aligns with current user familiarity on our cluster and ensures consistency with existing lab documentation.

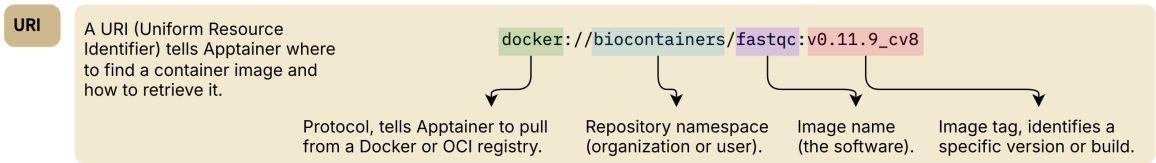
In [ ]:
```
module load singularity/4.3.4 ## latest version is generally recommended,
module list
```

In [ ]:
```
## Check the help message
singularity -h
```

Pull images

Download or pull a container from a given URI.

`singularity pull [output file] <URI>`



> **🛑 Critical: Manage Your Storage Quota**
> Please save your container images to your **group shared folder** instead of `$HOME`. Home directories are strictly capped at **30GB**. Large images will quickly trigger "Disk Quota Exceeded" errors, which can crash your session and prevent you from saving work.

In [ ]:
```
cd /cluster/tufts/yzhang85 # replace this directory to your own directory
mkdir containers
cd containers
```

Let's pull an image from Docker hub. In the demo, we will use tensorflow/tensorflow, the official tensorflow container built by the developer.

In [ ]:
```
singularity pull docker://tensorflow/tensorflow:latest
```

> **⚠ Warning: Avoid the 'latest' tag!**
> Using the *latest* tag is handy for quick typing, but dangerous for workflow

reproducibility. Under the hood, the *latest* tag can point to different image versions over time, making your scientific results difficult to replicate.

In [ ]:
```
ls
```

In [ ]:
```
## Let's create a python script to test the image
cat <<'EOF' > tf_test.py
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3' # Silent logs
import tensorflow as tf
import numpy as np

print("--- TensorFlow Vision Demo ---")

# 1. Load a pre-trained MobileNetV2 model (Lightweight & Fast)
# This model is trained to recognize 1,000 different objects.
model = tf.keras.applications.MobileNetV2(weights='imagenet')

# 2. Create a fake "Image" (a 224x224 RGB matrix)
# In a real workshop, you'd load a .jpg, but this works for a quick demo!
random_image = np.random.uniform(0, 255, (1, 224, 224, 3)).astype(np.float32)
preprocessed_image = tf.keras.applications.mobilenet_v2.preprocess_input(randc

# 3. Run Inference
print("Analyzing image...")
predictions = model.predict(preprocessed_image)
results = tf.keras.applications.mobilenet_v2.decode_predictions(predictions, t

print("\nTop 3 Predictions for this random data:")
for i, (imagenet_id, label, score) in enumerate(results):
    print(f"{i+1}. {label}: {score*100:.2f}%")

print("\nSUCCESS: TensorFlow loaded the model and performed inference!")
EOF
```

In [ ]:
```
ls -hl
```

singularity exec

`singularity exec [options] image command`

In [ ]:
```
singularity exec tensorflow_latest.sif python tf_test.py
```

How to run GPU containers

By default, a container is a closed box. It cannot "see" the graphics cards or the drivers installed on the host machine. `--nv` flag is the bridge between the isolated

container and the powerful NVIDIA GPUs on the cluster. You use it with the exec, run, or shell commands:

`singularity shell --nv pytorh_gpu.sif`

`singularity run --nv python_gpu.sif`

`singularity exec --nv pytorh_gpu.sif python torch_script.py`

🧹 Cache Management

When you pull or build images, Singularity and Apptainer store temporary "layers" and metadata in a hidden **cache** directory.

Why is this important?

- **Quota Issues**: By default, this cache is stored in your `$HOME` directory (`~/.singularity/cache`). Since home directories are capped at **30GB**, a few large pulls (like TensorFlow or PyTorch) can quickly trigger "Disk Quota Exceeded" errors.
- **Hidden Space**: Deleting a `.sif` file **does not** delete its cached layers. You must clean the cache manually to recover that space.

In [ ]:
```
# 1. Check how much space your cache is currently using
singularity cache list -v
```

In [ ]:
```
# 2. Clean CACHE dry-run
singularity cache clean --force
```

In [ ]:
```
# 3. Confirm cache has been cleaned
singularity cache list -v
```

Bind Mounting Host Directories

The Concept of Isolation

By default, a container is an isolated environment. It has its own internal file system and cannot "see" the files or directories on the HPC host (like your project space) unless specifically told to do so.

What is Bind Mounting?

Bind mounting acts as a bridge. It maps a directory on the host file system to a directory inside the container. This allows applications running inside the container to read from and write to the cluster's storage.

💡 **Tufts HPC Configuration: Automatic Mounting**
Inside our `singularity` and `apptainer` modulefiles, we have pre-configured common paths—such as `/tufts/cluster` and `/cluster/home`—to **mount automatically**. In most cases, you will not need to bind these manually. However, understanding the bind mechanism is vital for custom workflows or when accessing datasets outside these standard paths.

Syntax and Usage

To bind a directory, use the `--bind` or `-B` flag during the run, shell, or exec commands.

1. Short Syntax (Common) If you want the directory to have the same path inside the container as it does on the host: `singularity exec -B /cluster/tufts/lab_data my_container.sif ls /cluster/tufts/lab_data`

2. Long Syntax (Custom Mapping) If you want to map a host directory to a different path inside the container, use the `host_path:container_path format`:

Maps a long cluster path to a simple '/datasets' folder inside the container:

`singularity exec -B /cluster/tufts/biocontainers/datasets:/datasets my_container.sif ls /datasets`

Let's use the long syntax to mount `/cluster/tufts/biocontainers/datasets` into the container directory named `/datasets` and run `ls`.

In [ ]:
```
singularity exec -B /cluster/tufts/biocontainers/datasets:/datasets /cluster/t
```

GPU Homework (post-workshop practice)

⚠ Note: GPU-enabled containers can take a long time to pull and running them also requires a GPU node. For this reason, GPU examples will be assigned as **post-workshop homework**. Please complete the following steps on your own:

1. Log in to the cluster.
2. Start an interactive session on a GPU-enabled node.

`srun -N1 -n12 --gres=gpu:1 -p gpu -t0-2 --pty bash`

3. Pull a GPU-enabled container image (for example, a container with

PyTorch and CUDA support).

`cd /cluster/tufts/myLab/myFolder`

`module load singularity`

`singularity pull docker://pytorch/pytorch:2.10.0-cuda12.6-cudnn9-runtime`

4. Use the container image to run a simple PyTorch analysis on the GPU.

`singularity exec --nv pytorch_2.10.0-cuda12.6-cudnn9-runtime.sif python torch_test.py`

You can use my torch_test.py to run the pytorch test:

In [ ]:
```
import torch
import torch.nn as nn
import torch.optim as optim

# Basic info
print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if not torch.cuda.is_available():
    raise RuntimeError("CUDA is not available. GPU not detected.")

device = torch.device("cuda")
print("Using GPU:", torch.cuda.get_device_name(0))

# Create synthetic dataset
# y = 2x + 3 + noise
torch.manual_seed(42)
N = 10000

x = torch.randn(N, 1, device=device)
y = 2 * x + 3 + 0.1 * torch.randn(N, 1, device=device)

# Define a simple model
model = nn.Sequential(
    nn.Linear(1, 16),
    nn.ReLU(),
    nn.Linear(16, 1)
).to(device)

# Loss and optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training loop
epochs = 20
```

```python
for epoch in range(1, epochs + 1):
    optimizer.zero_grad()
    predictions = model(x)
    loss = criterion(predictions, y)
    loss.backward()
    optimizer.step()

    print(f"Epoch {epoch:02d} | Loss: {loss.item():.6f}")

# Inspect learned parameters
with torch.no_grad():
    for name, param in model.named_parameters():
        print(f"{name} mean value: {param.mean().item():.4f}")
        print("Training completed successfully on GPU.")
```

🎉 Congratulations! If your output is similar to this, you have successfully completed the container on HPC training and verified GPU execution with PyTorch.

Expected output:

PyTorch version: 2.10.0+cu126

CUDA available: True

Using GPU: NVIDIA H200

Epoch 01 | Loss: 8.657712

Epoch 02 | Loss: 8.218677

Epoch 03 | Loss: 7.782336

...

Epoch 18 | Loss: 2.200002

Epoch 19 | Loss: 1.943079

Epoch 20 | Loss: 1.707724

0.weight mean value: 0.3794

0.bias mean value: -0.0643

2.weight mean value: 0.1422

2.bias mean value: 0.3177

Training completed successfully on GPU.